

IRIS Studio: the cinema robot arm of the thesis as a browser simulator, with a servo zoom lens, a planned shot library and a learning loop

Jerry (Qilong) Cheng

September 2026

Abstract

IRIS is the six-axis cinema robot arm of the author’s MEng thesis [1]: Unitree GO-M8010-6 actuators, an offset-axis layout, 0.7 m of reach and a visuomotor policy that learned camera moves from demonstrations. This document records the studio built on top of it in September 2026: the thesis’s MuJoCo kinematic model driven in the browser, a desk with product sets, a Sony α 7R III with a 16–150 mm zoom whose zoom and focus rings are servo-driven, a thin-lens depth-of-field model of the feed, 23 preset shots (pans, dollies, orbits, arcs, dolly zooms, rack focus, object tracking) defined as parametric lens paths, a damped-least-squares solver with a tool frame at the lens’s entrance pupil, and a demonstration-recording and policy-training loop. Every preset is planned at 30 Hz: the maximum joint speed over the library is $122^\circ/\text{s}$ and the peak inverse-dynamics torque 14.6 N m against the motors’ 23.7 N m, with 0 unreachable samples. The dolly zooms hold the subject’s silhouette width to a coefficient of variation of 0.1 % and 0.1 % where a plain dolly changes it by 14.6 %, and a pixel measurement off the rendered clips confirms a 2.4 % band against 38.5 %. The videos on the web page were recorded from the same simulator; the executed telemetry (5739 frames over 23 shots) shows a mean lens position error of 0.26 mm and a median aim offset of 0.05 % of the frame. All figures are regenerated by `experiments/make_figures.py`.

Keywords: cinema robotics, inverse kinematics, camera control, zoom lens, dolly zoom, imitation learning, browser simulation.

1 Overview

The thesis built IRIS, characterised it and learned camera moves from demonstration (VisionJoint-Planner: a five-frame window of RGB and joint states and a goal point in, the next five joint states out, trained on 17,897 samples). Its simulation lived in MuJoCo on a workstation and its camera was a RealSense. The studio described here moves the arm to the web page `iris.html` and its full-screen app `iris_studio.html`, adds the camera the next IRIS will carry, and turns cinematography into a library of planned, validated presets that a visitor can play, edit and learn from. The Python side (`experiments/irisstudio`) and the JavaScript side (`assets/js/iris`) implement the same kinematics and the same shot rules; the planner validates in Python what the studio executes in the browser, and the browser’s own execution is logged back for Section 3.4.

2 Method

2.1 Kinematics from the MuJoCo tree

`iris_model.json` is exported from `mujoco_sim/assets/iris.xml`: for every body its fixed pose in the parent (position, quaternion), its hinge joint (axis, range) and its mesh. MuJoCo composes $T_{\text{world},b} = T_{\text{world,parent}} T(\mathbf{p}_b, \mathbf{q}_b) R(\hat{\mathbf{a}}_b, q_j)$, and both the Python and the JavaScript forward kinematics do the same; `tests/test_kinematics.py` checks positions and rotations of four bodies against `mj_forward` to 10^{-9} over 50 random configurations, and the geometric Jacobian against `mj_jacBody`. Joint 1 at zero puts the arm’s plane along $+y$; -90° faces the products. Joints 4 ($\pm 180^\circ$) and 6 ($\pm 360^\circ$) are continuous-rotation motors: the solver leaves them unclamped and every path difference is taken modulo 2π (an early planner counted a $\pm 180^\circ$ wrap as a 358° jump).

The tool frame. The mount’s $+z$ is the optical axis and its $-y$ is up (the convention of the thesis’s look-at controllers). The camera body and lens are modelled to scale; the lens axis passes through the mount origin and the entrance pupil sits $\mathbf{o} = (0, 0, 0.14\text{m})$ ahead of it, so the tool frame is the mount frame translated by $R\mathbf{o}$. A first version aimed the mount origin and put the lens 2cm beside it, which placed the subject 10% off centre at close range; aiming the tool frame removed it. All shot positions are lens positions.

2.2 Inverse kinematics

Given a target lens position $\mathbf{p}^*$ and rotation R^* (a look-at rotation with a level horizon, or yaw/pitch/roll for pans and tilts), damped least squares iterates

$$\Delta \mathbf{q} = J^\top (JJ^\top + \lambda I)^{-1} \mathbf{e}, \quad \mathbf{e} = [\mathbf{p}^* - \mathbf{p}(\mathbf{q}); w \log(R^* R(\mathbf{q})^\top)^\vee], \quad \lambda = 10^{-3}, w = 0.3, \quad (1)$$

with a step of 0.6, at most 80 iterations from the previous joint vector, and a tolerance of 0.1 mm and 0.06° . When the warm start fails (edge of reach), eight canonical postures are tried and the solution closest to the previous joint vector is kept; without a previous vector, the one whose base yaw faces the target with the largest margin to the joint limits. Unreachable poses make the studio hold its last pose and flag it, instead of clamping into a folded posture as an early build did. A per-joint speed cap of 1.5 rad/s (3 rad/s for the whip pan) filters the commanded joints; this is the “cinematic” rate limiter of the thesis’s deployment stack, not a dynamic model.

2.3 The lens

Full-frame sensor $35.9 \times 24\text{mm}$. Field of view $2 \arctan(24/2f)$ vertical. Depth of field from the thin-lens circle of confusion,

$$c(d) = \frac{f}{N} \frac{|d - S|}{d} \frac{f}{S - f}, \quad \text{acceptable } c = 0.03\text{mm}, \quad (2)$$

rendered in the feed by a Poisson-disk gather whose radius is c converted to pixels, with each tap weighted by its own circle so that sharp foregrounds do not bleed. Zoom and focus rings are servos slewing at 60 mm/s and 4 m/s with a short first-order lag; the barrel extends 35 mm over the zoom range. The dolly zoom holds the subject’s size in the frame while the arm moves the lens along the optical axis. The textbook law $f(t) = f_0 d(t)/d_0$ does that for a point at the subject distance,

but what the audience watches is a silhouette: a convex body of radius r at distance d subtends a half-angle $\arcsin(r/d)$, so its image half-width is $f r / \sqrt{d^2 - r^2}$ and holding *that* fixed requires

$$f(t) = f_0 \frac{\sqrt{d(t)^2 - r^2}}{\sqrt{d_0^2 - r^2}}, \quad (3)$$

which collapses to the textbook law as $r \rightarrow 0$. The difference is not academic at these distances: on the 0.225 m carrot cake between 0.26 m and 0.42 m, the point law leaves about 7 % of breathing in the silhouette. The size that enters r is the object’s measured extent in the scene (`experiments/measure_products.js`); the spec’s `height` field is metadata for the pedestal offset and is not the object’s width — the cake is 98 mm tall and 225 mm across, and framing on the wrong one put it at 1.5 frame widths, running off both sides for the whole shot. The planner tracks the silhouette width for every shot (Figure 5).

2.4 Shots

A preset is a path (static, line, horizontal orbit, vertical arc), an aim (look-at a subject or point, look-at moving between two points, yaw/pitch/roll ranges, or track the car), a lens track (focal length constant, ramped or dolly-zoom; focus automatic, fixed, ramped or a rack between subjects; aperture) and a duration with an easing (smoothstep, linear, or an exponential snap for the whip pan). Subjects are roles (*near* hero at $x = 0.32$ m, *far* at 0.74 m, extras, the car) resolved by the active product set, so every preset works on every set. The toy car drives a 0.18 m circle around (0.64, 0) in 12 s; the tracking preset aims at its true position, and the studio offers a pixel tracker (red-blob centroid on a 64×43 read-back of the feed, proportional gaze control) as the alternative.

2.5 The planner

`plan_shots.py` samples each preset at 30 Hz, solves the IK along it, unwraps the continuous joints, differentiates for joint speeds and accelerations, and calls `mj_inverse` on the thesis model (contacts disabled; the 1.5 kg camera payload is the model’s own `ee_mount` mass) for the joint torques. It reports per shot the IK residuals, the maximum joint speed and which joint, the peak torque and which joint, the closest approach to a joint limit, the lens ranges and the subject-size statistics, and exports the joint tracks for the page. Presets that failed the first drafts were re-sited with small parameter sweeps (Section 5).

2.6 Recording and learning

The studio records episodes at 10 Hz: joints, focal length, focus distance and a 24×16 grey thumbnail of the feed, tagged with the shot and the product positions. The in-tab policy is a VisionJointPlanner-lite in TensorFlow.js: five past states and the thumbnail through a two-layer CNN, a goal token (shot one-hot plus the hero and far positions), a 256-wide MLP with layer normalisation, and the next five joint and lens deltas (action chunking); at run time ACT-style temporal ensembling averages the chunks predicted at previous steps with weights $e^{-0.35j}$.

2.7 Video capture

`record_videos.js` drives the app in headless Chrome on the GPU through puppeteer, steps the simulation deterministically at 30 fps through `window.IRIS.stepFrame`, screenshots each frame of

the film layout (studio view left, feed right, 1920×1080), encodes with `ffmpeg`, and stores the per-frame telemetry (joints, lens, IK status and residual, the aim point's position in the frame) that Section 3.4 analyses.

3 Experiments

3.1 Lens model

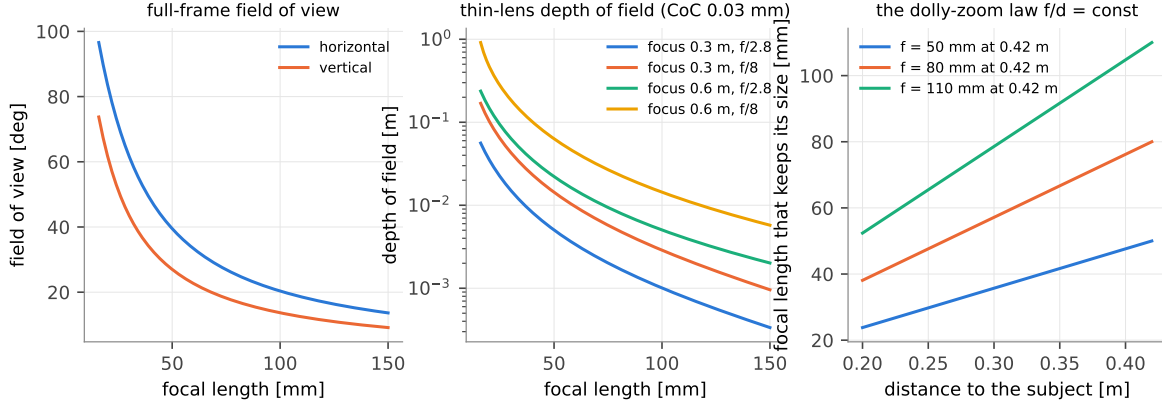


Figure 1: Field of view over the zoom range; depth of field for two focus distances and apertures; and the dolly-zoom law.

3.2 Workspace and the shot library

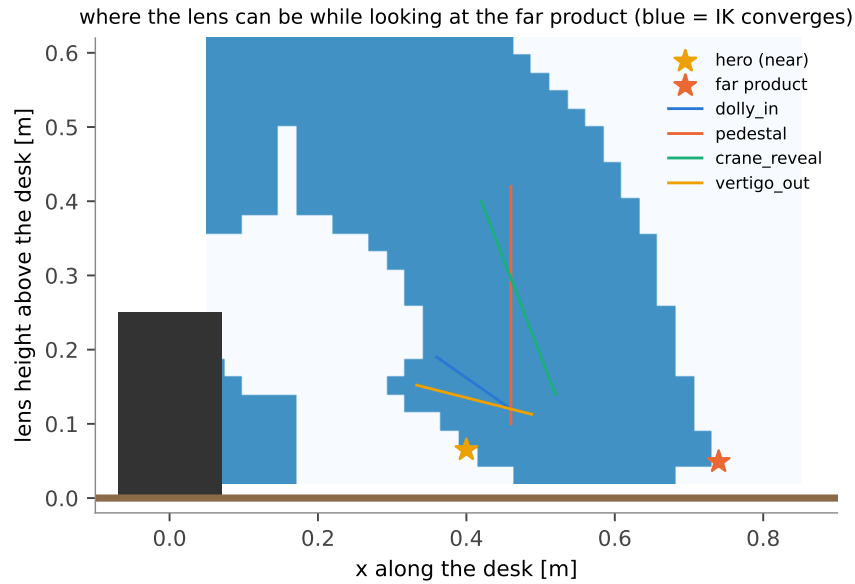


Figure 2: Lens positions in the vertical plane of the desk from which the far product can be looked at (IK converges), with four planned paths.

Table 1: Every preset through the planner (30 Hz): duration, focal length, maximum joint speed, peak inverse-dynamics torque, IK residuals, and the closest approach to a joint limit (continuous joints excluded).

shot	s	mm	$\dot{q}_{\max}$ [°/s]	$\tau_{\max}$ [N m]	pos [mm]	rot [°]	limit [°]
Pan	5	28	40	10.2	0.07	0.06	16
Tilt	5	28	29	7.9	0.10	0.06	15
Dutch roll	5	40	19	5.9	0.07	0.06	9
Zoom in	6	24–110	0	5.8	0.06	0.00	2
Zoom out	6	24–110	0	5.8	0.06	0.00	2
Dolly in	6	35	8	9.6	0.10	0.01	5
Dolly out	6	35	8	9.8	0.10	0.02	5
Truck (lateral)	6	35	100	14.6	0.10	0.02	25
Pedestal (crane)	6	35	42	11.0	0.10	0.02	25
Orbit 90°	8	35	65	5.6	0.10	0.05	15
Orbit 180°	14	35	75	10.3	0.10	0.05	11
Turntable 360°	14	45	0	5.9	0.06	0.01	29
Top view to side view	8	30	14	13.5	0.10	0.02	6
Crane reveal	7	28	31	10.3	0.10	0.03	37
Dolly zoom (out + zoom in)	8	24–42	7	10.3	0.10	0.01	4
Dolly zoom (in + zoom out)	8	24–42	7	10.1	0.10	0.01	4
Rack focus	8	55	0	3.9	0.10	0.01	9
Object tracking	12	45	46	6.1	0.10	0.06	10
Follow the car	12	40	55	6.4	0.10	0.06	5
Low reveal	6	24	63	8.7	0.10	0.06	19
Whip pan	4	35	122	13.2	0.09	0.06	13
Arc + zoom	12	30–60	83	6.4	0.10	0.05	2
Hero push-in	6	40–60	16	12.4	0.10	0.06	27

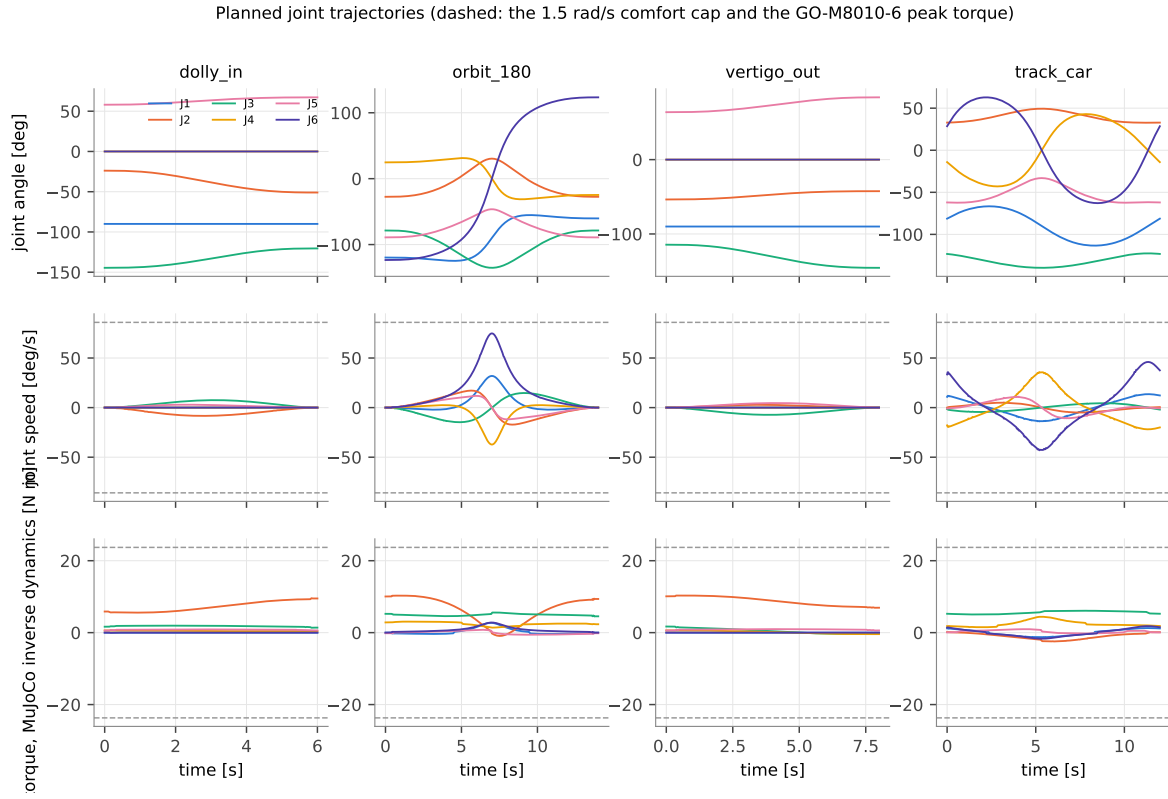


Figure 3: Four presets: joint angles, speeds against the 1.5 rad/s cap, and torques against the 23.7 N m peak.

Every preset through the planner: within the speed cap, the torque limit, and clear of the joint limits

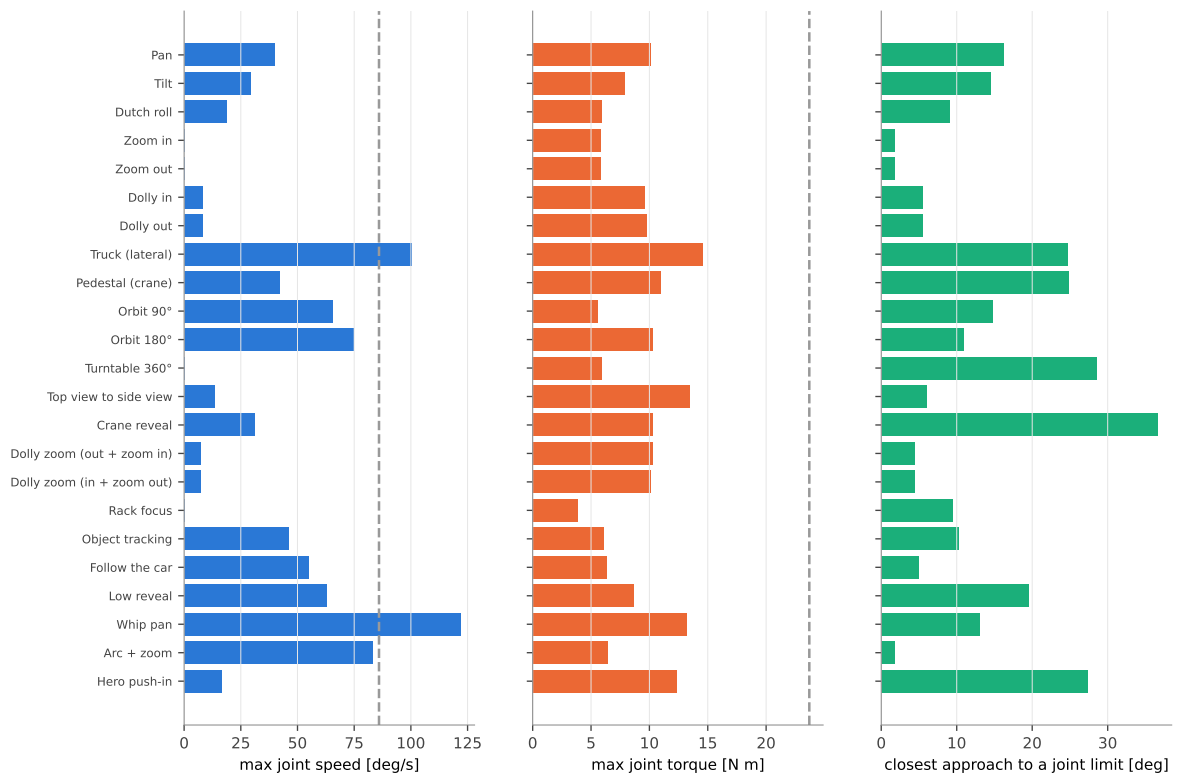


Figure 4: Per-shot maxima over the library.

Table 1 is the library as shipped: 23 presets, 178 s of motion, 0 unreachable samples, 2 presets above the comfort cap (the whip pan, by design), the largest speed $122^\circ/\text{s}$ and the largest torque 14.6 N m, at the shoulder as the thesis’s sizing predicts (25.8 N m worst case static with a 1.5 kg payload at full extension, which no preset reaches).

3.3 Dolly zoom

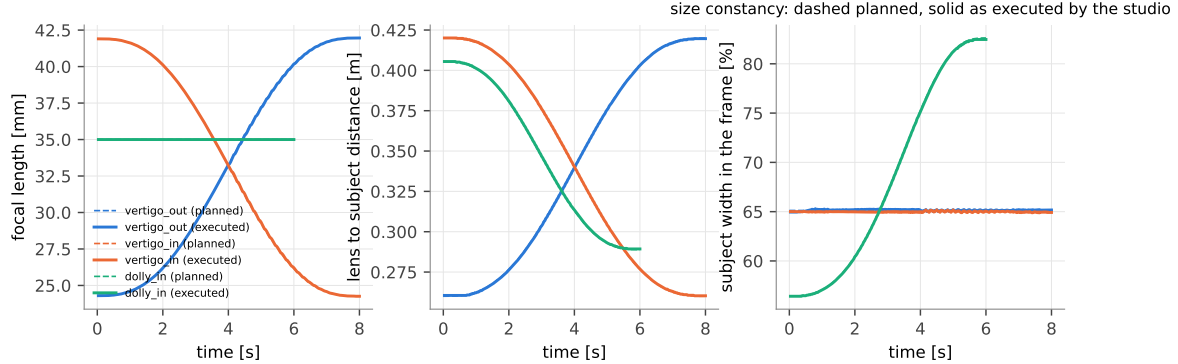


Figure 5: Focal length, lens distance and the subject’s silhouette width along the two dolly zooms and a plain dolly.

Both dolly zooms travel from 0.26 m to 0.42 m along the optical axis, 24 mm to 42 mm at $f/16$, with the cake at 65 % of the frame width. As executed — servo lag, joint tracking and the discrete focus of the lens model included — the subject’s width in the frame varies by 0.1 % (out + zoom in) and 0.1 % (in + zoom out), against 14.6 % for the plain dolly over the same travel.

That number is still the model checking itself. As an independent test, `experiments/measure_size_constancy.py` segments the cake out of the rendered camera feed of each recorded clip (warm, mid-dark connected components, the thin HUD rules rejected by area) and measures its silhouette in pixels: median 698 px with a 5–95 % spread of 2.4 % for the dolly zoom in, 700 px and 2.3 % out, against 888 px and 38.5 % for the plain dolly (Figure 6). The residual few per cent is real and unavoidable: the cake is not a sphere, so its silhouette is not exactly the tangent circle Equation 3 assumes, and its top face foreshortens as the lens rises. The occasional single-frame outlier is the segmentation losing a fragment where the specular band crosses the cake.

3.4 Executed telemetry

Over 5739 recorded frames of 23 shots the live solver reports a mean position error of 0.26 mm (maximum 2.0 mm at the fast whip) and 0 frames out of reach; the aim point sits 0.05 % of the frame from the centre (median per shot, averaged over shots; the maximum, 96.6 %, is the whip pan’s snap), which is the difference between the commanded look-at and the rate-limited joints. 60 frames whose aim point lay outside the frustum while the arm settled into its first pose are excluded.

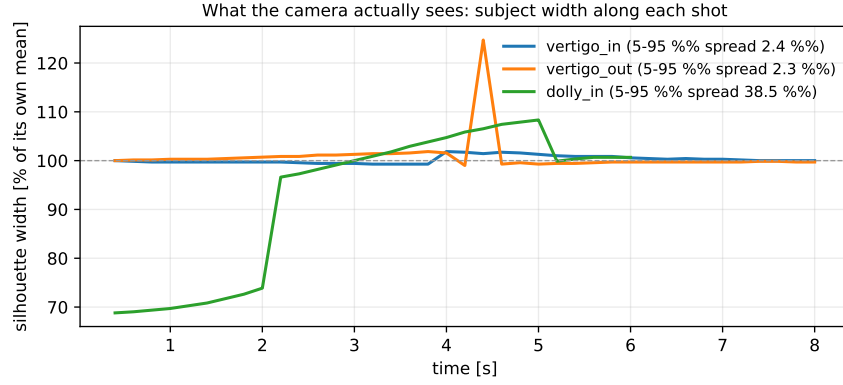


Figure 6: Subject silhouette width measured in pixels from the rendered clips, as a percentage of each clip's own median.

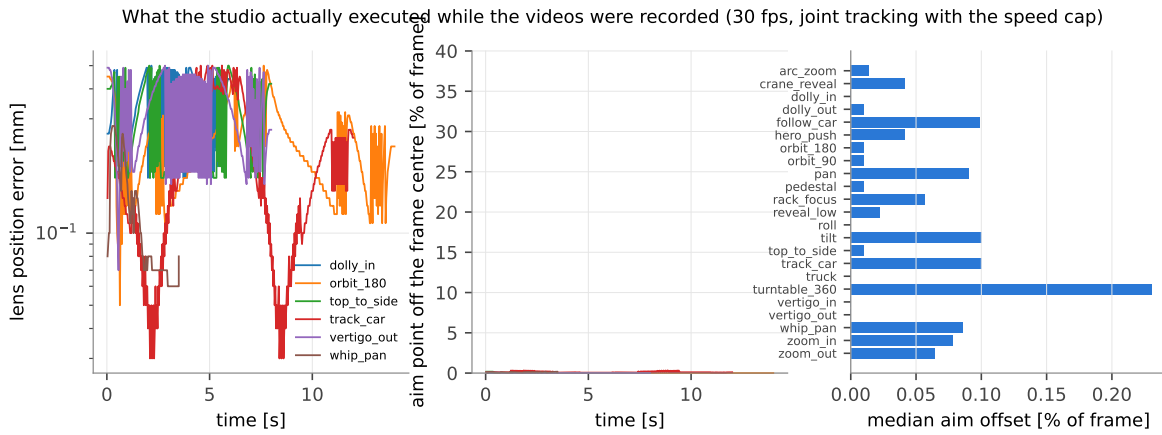


Figure 7: From the video recordings: the live solver's lens position error and the aim point's offset from the frame centre.

4 Implementation and usage

experiments/irisstudio/kinematics.py (Arm, tool frame, IK, seeds), shots.py (the evaluator), plan_shots.py, tune_vertigo.py (re-sites the dolly zooms against IK, lens range and depth of field), measure_products.js (the products' real extents), record_videos.js, measure_size_constancy.py (the pixel check of Figure 6), make_figures.py; the spec out/iris_studio.json is copied to the site as assets/data/iris_studio.json. Browser: assets/js/iris/kin.js, lens.js, shots.js, scene.js (three.js: desk, arm from the decimated STL links, camera rig, products, lights), dof.js, policy.js, app.js; pages iris_studio.html and iris.html.

```
cd experiments && python -m pytest -q tests
node measure_products.js          # out/product_sizes.json (real extents)
python tune_vertigo.py --apply    # re-sites the two dolly zooms
python plan_shots.py              # out/shots_plan.json (-> assets/data/)
node record_videos.js             # out/video/iris_<shot>.mp4 + telemetry
python measure_size_constancy.py  # pixel check on the clips
python make_figures.py           # figures, numbers.tex, shots_table.tex
cd ../docs && make
```

The two node steps need python3 -m http.server 8765 running in the site root. Hardware path: the planner's joint tracks are 30 Hz position setpoints in the model's joint order; the thesis's ROS node consumes exactly that over RS-485, and the lens tracks are two servo setpoints per frame.

5 Lessons and limitations

- Orbits at desk height fail: looking back toward the base needs the wrist to fold past its $\pm 100^\circ$ pitch. The turntable orbit lives high above the product and looks down; half orbits stay on the base side.
- A straight-down top view sits on the look-at singularity (the horizon has no meaning); the top-to-side arc starts at 70° elevation and comes in from the side.
- Any lens position closer than about 0.3 m to the base column folds the arm onto itself once the tool offset is included; the first drafts of the dollies started there.
- The wrist singularity ($q_5 \rightarrow 0$) makes joints 4 and 6 swing quickly when the optical axis passes through the forearm axis; the tracking presets were re-sited to keep $|q_5|$ away from zero, and continuous joints are unwrapped.
- Not modelled: motor dynamics and belt compliance, self-collision, lens breathing and the real minimum-focus curve, exposure. The in-tab policy is a toy; the thesis's numbers stand.

References

- [1] Q. Cheng, "IRIS: Intelligent Robotic Imaging System," MEng thesis, University of Toronto, 2025.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2005.

- [3] S. R. Buss, “Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods,” technical report, UCSD, 2004.
- [4] T. Z. Zhao, V. Kumar, S. Levine and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *RSS*, 2023.
- [5] E. Todorov, T. Erez and Y. Tassa, “MuJoCo: A physics engine for model-based control,” *IROS*, 2012.
- [6] M. Potmesil and I. Chakravarty, “A lens and aperture camera model for synthetic image generation,” *SIGGRAPH*, 1981.